

App Dev 2 (TMS) Report

1. Student Details

Name: Naina Bhatt

Roll Number: 24f3001863

Email: 24f3001863@ds.study.iitm.ac.in

I am currently in the second term of my diploma at the IIT Madras BS Degree, where I am building my skills in web and software development through hands-on projects. I am particularly interested in learning full-stack development, APIs, databases, and modern web technologies.

2. Project Details

Project Title: Trekkify – Trekking Management Application

Problem Statement:

To design and develop a web-based application for managing trekking activities, trek bookings, users, and trek operations through a centralized platform. The application supports three major roles: Admin, Trek Staff, and Trekker, with role-based access to their respective functionalities.

Approach:

Trekkify is developed as a full-stack web application using Flask REST APIs for the backend and Vue.js for the frontend.

Key functionalities include:

- Separate functionality for Admin, Trek Staff, and Trekker users using JWT authentication.
- Admins can create, update, and manage trekking activities.
- Trekkers can browse available treks and book them.
- A trekker cannot book the same trek multiple times.
- Trek staff can view their assigned treks and registered participants.
- Staff can update available slots and trek status.
- Role-specific dashboards provide relevant trek and booking information.
- Redis is used to cache frequently accessed data.
- Celery and Celery Beat are used for asynchronous and scheduled tasks.

- Frontend and backend validation are implemented to prevent invalid operations.
-

3. AI/LLM Declaration

I used ChatGPT (GPT-5) as an assistance tool during the development of the project for understanding technical concepts, improving code structure, guidance and documentation. The final implementation, integration, testing, and project decisions were performed by me.

AI/LLM usage: Below 30% in the overall project.

4. Technologies and Frameworks Used

Database

- SQLite: Lightweight relational database
- Flask-SQLAlchemy: ORM for database interaction

Backend

- Flask: Backend framework for developing REST APIs
- Flask-JWT-Extended: JWT authentication & authorization

Frontend

- Vue.js 3(CLI): Frontend framework
- Vite: Frontend development & build tool
- Axios: Communication between frontend and backend
- Bootstrap 5: Responsive UI styling

Background Jobs & Caching

- Redis: Caching and Celery message broker
 - Celery: Scheduled and async background tasks
 - Celery Beat: Periodic task scheduler
-

5. Database Schema / ER Diagram

The application uses **SQLite** as its relational database, with **Flask-SQLAlchemy** used as the ORM for creating models and performing database operations. The database is designed to

maintain user accounts, trek information, bookings, staff details, and their relationships in a structured manner.

Main Tables:

1. User

Stores user account and authentication information.

The role identifies whether the user is an **Admin, Trek Staff, or Trekker**.

- id
- username
- name
- email
- password
- role

2. Trek

Stores information about each trekking activity, including trek details, available slots, trek status, and the staff member assigned to manage the trek.

3. Booking

Stores booking records created by trekkers. It maintains details such as the user, trek, booking status, and payment status. A user cannot create multiple bookings for the same trek. .

4. StaffProfile

Stores additional information specific to trek staff and is associated with the corresponding user account.

Relationships

- User → Booking: One-to-Many
- Trek → Booking: One-to-Many
- User → StaffProfile: One-to-One
- User → Trek: One-to-Many through assigned staff
- User + Trek → Booking: Used to prevent duplicate bookings

ER Diagram:



Figure: ER Diagram of the Trekkify database

6. API Resource Endpoints

User APIs

POST	<code>/register</code>	Register a new user
POST	<code>/login</code>	Authenticate user
GET	<code>/profile</code>	View user profile
PUT	<code>/profile</code>	Update user profile

Admin APIs

GET	<code>/admin/dashboard</code>	View admin dashboard
GET	<code>/admin/users</code>	View all users
GET	<code>/admin/users/<id></code>	View a specific user
PUT	<code>/admin/users/<id></code>	Update user details
DELETE	<code>/admin/users/<id></code>	Delete a user
GET	<code>/admin/treks</code>	View all treks
POST	<code>/admin/treks</code>	Create a trek
PUT	<code>/admin/treks/<id></code>	Update trek details
DELETE	<code>/admin/treks/<id></code>	Delete a trek

PUT	<code>/admin/treks/<id>/assign-staff</code>	Assign staff to a trek
-----	---	------------------------

GET	<code>/admin/bookings</code>	View all bookings
-----	------------------------------	-------------------

Trekker APIs

GET	<code>/treks</code>	View available treks
-----	---------------------	----------------------

GET	<code>/treks/<id></code>	View trek details
-----	--------------------------------	-------------------

POST	<code>/bookings</code>	Book a trek
------	------------------------	-------------

GET	<code>/bookings</code>	View user's bookings
-----	------------------------	----------------------

GET	<code>/bookings/<id></code>	View booking details
-----	-----------------------------------	----------------------

PUT	<code>/bookings/<id></code>	Update booking
-----	-----------------------------------	----------------

DELETE	<code>/bookings/<id></code>	Cancel a booking
--------	-----------------------------------	------------------

Trek Staff APIs

GET	<code>/staff/dashboard</code>	View staff dashboard
-----	-------------------------------	----------------------

GET	<code>/staff/treks</code>	View assigned treks
-----	---------------------------	---------------------

GET	<code>/staff/treks/<id></code>	View assigned trek details
-----	--------------------------------------	----------------------------

PUT	<code>/staff/treks/<id>/slots</code>	Update available slots
-----	--	------------------------

PUT	<code>/staff/treks/<id>/status</code>	Update trek status
GET	<code>/staff/treks/<id>/participants</code>	View registered participants

7. Architecture and Features

Architecture Overview

Trekkify follows a client-server architecture.

- Vue.js + Vite: Frontend application
- Axios: Sends API requests
- Flask: REST API backend
- SQLAlchemy: Database ORM
- SQLite: Persistent data storage
- Redis: Caching and message broker
- Celery: Background task execution
- Celery Beat: Periodic task scheduling

Implemented Features

Authentication and Authorization

- User registration and login
- JWT-based authentication
- Role-based access control
- Separate permissions for Admin, Trek Staff, and Trekker

Trekker Features

- Browse available treks
- View trek details
- Book treks
- View bookings
- Manage booking status

- Duplicate booking prevention

Admin Features

- Manage users
- Manage treks
- Create and update trek information
- Assign trek staff
- Manage trek slots
- Monitor bookings

Trek Staff Features

- View assigned treks
- View registered trekkers
- Manage participants
- Update available slots
- Update trek status
- Manage trek operations

Redis and Celery

Redis is used for caching frequently accessed data and as the message broker for Celery. Celery handles asynchronous background tasks, while Celery Beat schedules periodic tasks.

7. Video link:

Presentation Video:  MAD 2 Project Video Presentation (TMS) - Naina Bhatt.mp4